In [ ]:
```
import math
```

Calculation of Orientation & Displacement for Goal Position

In [ ]:
```
def calculate_movement_and_turns(current_pos, current_orientation, goal_pos, goal_orientation):
    # Extract current and goal positions
    x, y = current_pos
    theta = current_orientation
    x_goal, y_goal = goal_pos
    theta_goal = goal_orientation

    # Calculate angle to the goal position
    angle_to_goal = math.degrees(math.atan2(y_goal - y, x_goal - x))

    # Calculate the initial turn angle
    initial_turn_angle = angle_to_goal - theta
    # Normalize the angle to [-180, 180] range
    initial_turn_angle = (initial_turn_angle + 180) % 360 - 180

    # Calculate the distance to the goal position
    distance_to_goal = math.sqrt((x_goal - x) ** 2 + (y_goal - y) ** 2)

    # Calculate the final turn angle to match the goal orientation
    final_turn_angle = theta_goal - angle_to_goal
    # Normalize the angle to [-180, 180] range
    final_turn_angle = (final_turn_angle + 180) % 360 - 180

    return initial_turn_angle, distance_to_goal, final_turn_angle
```

In [ ]:
```
# Example usage
current_pos = (1,1)
current_orientation = 45
goal_pos = (2,5)
goal_orientation = 0

initial_turn, distance, final_turn = calculate_movement_and_turns(current_pos, current_orientation, goal_pos,
print(f"Initial Turn Angle: {initial_turn} degrees") #alpha
print(f"Distance to Move: {distance}")
print(f"Final Turn Angle: {final_turn} degrees")
```

```
Initial Turn Angle: 30.963756532073546 degrees
Distance to Move: 4.123105625617661
Final Turn Angle: -75.96375653207353 degrees
```

Code for turning the robot

In [ ]:
```
def turn_to_angle(alpha):
    l_aw = a_w
    a_w = (a_w + 180) % 360 - 180
    w_diff = (l_aw - a_w)

    if alpha>0:
        cmd = 3 #turn left
    elif alpha<0:
        cmd = 4 # turn right
    else:
        pass

    send_command(cmd) #i2c command

    while(w_diff < abs(alpha)):
        a_w = (a_w + 180) % 360 - 180
        w_diff = (l_aw - a_w)


    cmd = 5 #stop
    send_command(cmd)
    update_orientation(alpha)
```

In [ ]:
```
def update_orientation(alpha):
    p_w = p_w + alpha
```

Code for displacing the robot

In [ ]:
```
def disp_to_dist(k):
    l_x,l_y = a_x, a_y
    dist = math.sqrt((a_x-l_x)**2 + (a_y-l_y)**2)
    l_px, l_py = p_x,p_y

    cmd = 1 #forward
    send_command(cmd)

    while((distance < k) and not(Obs)):
        dist = math.sqrt((a_x-l_x)**2 + (a_y-l_y)**2)
        Obs = check_obs()
        update_coord(dist,p_w)

    cmd = 5
    send_command(cmd)
```

In [ ]:
```
def update_coord(dist,p_w):
    p_x = l_px + dist * math.cos(p_w)
    p_y = l_px + dist * math.sin(p_w)
    p_w = p_w
```

- a_x,a_y,a_w is the actual odemetry data comming form camera(realtime data from call back function)
- p_x,p_y,p_w is the assumed starting point & orientation which is initially (0,0,0)
- this code works by measuring orientation difference while turning and measuring distance while moving forward.
- This code is untested and should work after some modification irrespective of whatever be the actual coordinates are.

Sequence of the functions should be

In [ ]:
```
initial_turn, distance, final_turn = calculate_movement_and_turns(current_pos, current_orientation, goal_pos,
turn_to_angle(initial_turn)
disp_to_dist(distance)
```